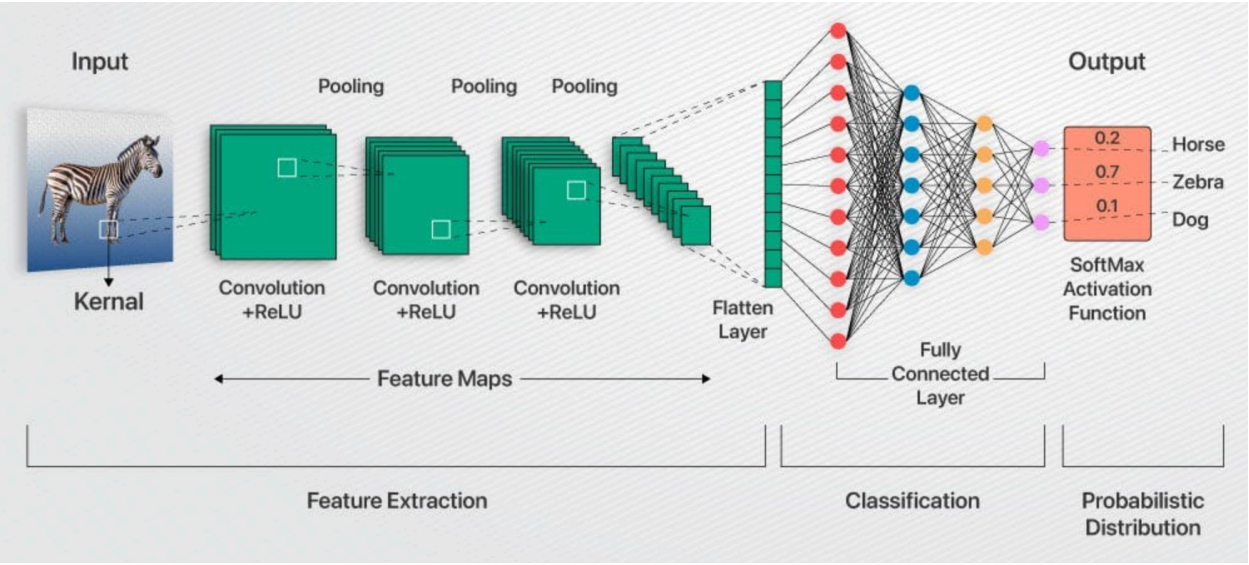
CNNs and based architectures:



An advancement from deep neural networks, convolutional neural networks perform convolution where as in dnns, there is matrix multiplication where there is an entire matrix of size output pixels x input pixels, multiplied to the input. In convolution as well there is a matrix of that size, but most of it's entries are 0 (sparse rep), hence reducing the need for such a big matrix.
Moreover the row elements of the matrix are the same, this shared parameter model helps the network learn a particular feature present in the image. Thus this operation resembles a convolution where a smaller weight matrix or a kernel is slid over the input image and accordingly an output image is created. Thus not every input pixel influences every output pixel in the output map, but as we progress towards deeper layers, the receptive field includes almost all the original inputs, hence are influenced by them. This sparse representation, parameter sharing and translational invariance helps convolutions perform significantly better than dnns in terms of efficiency and robustness for feature extraction. These features can further be used efficiently in classification, segmentation and detection tasks. (by being made smaller and smaller)

Another advantage posed by CNNs is their pooling layer, that after the non linear activation layer is used to further reduce the output size by maxpooling, avg pooling etc. These allow the output feature map to be invariant to transformations such as translation, minor rotations, or small distortions as the pooled representations are largely the same or pooling summarizes the responses over a whole neighborhood. It encourages generalization by focusing on whether a feature exists, not exactly where.
Pooling across multiple kernel outputs allows the network to learn which transformations are redundant by combining similar features. This lets the network focus on the existence of features rather than the precise variant, improving generalization and efficiency eg Imagine 3 filters detecting

a line at slightly different angles.If you pool across these feature maps, the network can learn invariance to the type of transformation the different filters represent.
Max-pooling across the filter outputs: you get one strong activation whenever the line appears, regardless of its exact orientation.This is invariance to the rotation transformation represented by the multiple filters.

In general, Pooling over spatial regions produces invariance to translation ,but if we pool over the outputs of separately parametrized convolutions, the features can learn which transformations to become invariant to.

How the image shown above works:
CNN takes an image as input and passes it through multiple layers, gradually learning to recognize important features. Early convolutional layers detect low-level, local patterns such as edges and corners, which are present in nearly all images and are not very discriminative. Pooling layers reduce spatial dimensions, remove redundancy, and introduce translation invariance. Deeper convolutional layers combine the feature maps from earlier layers to form more abstract, higher-level patterns and shapes over a larger receptive field, capturing more context and task-relevant information. As the network goes deeper, filters in higher layers hierarchically aggregate the outputs of previous layers, enabling the detection of complex features—like object parts or entire objects—that are most useful for classification. This hierarchical abstraction summarizes the global context of the image while retaining the most relevant features, which enhances generalization. The final convolutional feature maps encode distributed patterns—a combination of many lower-level features—rather than visually interpretable parts, reflecting the statistical patterns that best distinguish classes for the network's task. These feature maps are then flattened and passed through fully connected layers to make the final prediction. Studies (.https://arxiv.org/pdf/1311.2901) have shown that deeper layers are visually more important for distinguishing objects, demonstrating that CNNs progressively learn complex, discriminative patterns, such as ears, eyes, or abstract representations, even if these are not directly interpretable to humans.

RESNETS, U-NETS,MOBILENETs, INCEPTIONNETs

RESNETs https://arxiv.org/pdf/1512.03385v1

The ResNet Models are a type of deep convolutional neural network models that use skip connections to train deeper networks effectively. Ok we've all heard that definition, it either sounds like jargon or something trivial. Let's dive deeper

These networks were mainly developed to ease the difficulty in training very deep CNNs. The main limitation of very deep CNNs was the problem of vanishing/exploding gradients which was solved using normalization, the limitation ResNet tried tackling was degradation.
Degradation is when the **training** loss and accuracy becomes worse as the number of layers increase. In the case of overfitting, training loss should decrease and vanishing gradient was ruled out as this occurred even in the case where batch normalization was present in the network.

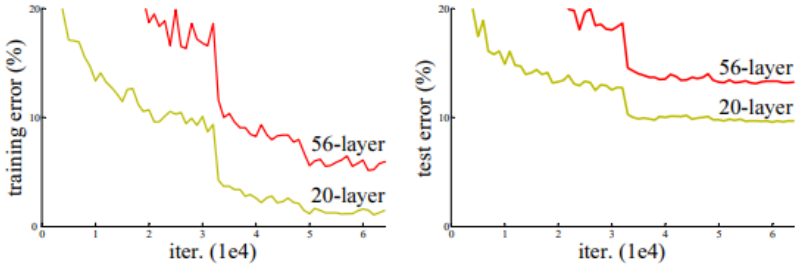


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer "plain" networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Degradation of training accuracy shouldn't occur intuitively, consider a shallow network and a deeper network made by adding more layers to the shallow network. In the worst case, these extra layers should just learn the identity function and the deeper network should perform just as good as the shallower network. But this does not occur (at least in feasible time) as learning the identity function using a bunch of non linear layers is extremely difficult and moreover the loss surface is quite complex due to the number of params involved in learning H(x) (the required function), which is why the authors of ResNet introduced the skip connection or the identity short cut as they call it.

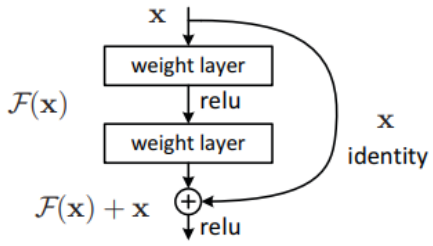


Figure 2. Residual learning: a building block.

This enables the network to learn the residual function H(x) - x, where x is the input and H(x) is the desired function, which is hypothesized to be easier to learn, for example it is easier to learn F(x) = 0 than F(x) = x, easy in terms of convergence rate. These results have been empirically verified in the paper.
In case the input has a different size when compared to the output, the shortcut connection has a weight matrix and the shortcut is called a projection shortcut rather than an identity shortcut due to the projection onto another dimensional space.

Thus skip connections allow deeper CNNs to accurately describe the function, due to the the proper utilization of more number of parameters and easier learning of the function, also helping in preventing vanishing gradients and significant forward propagating values.

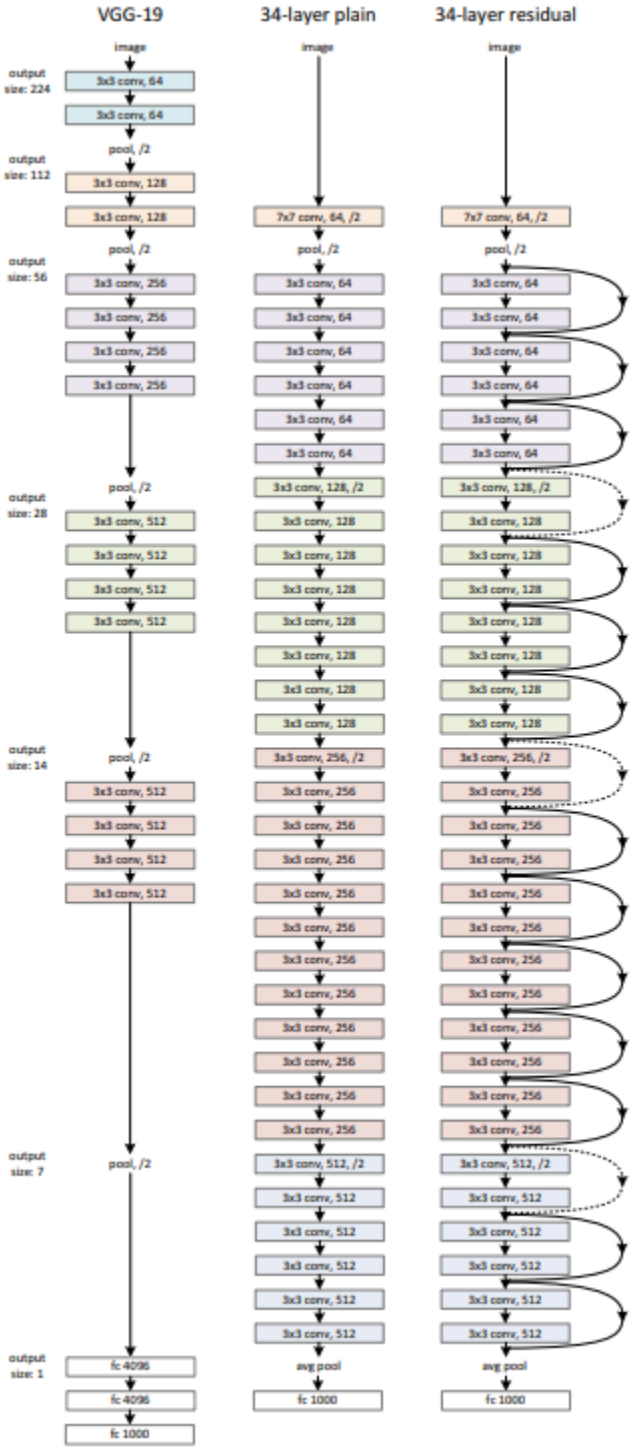
method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43†
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	**19.38**	**4.49**

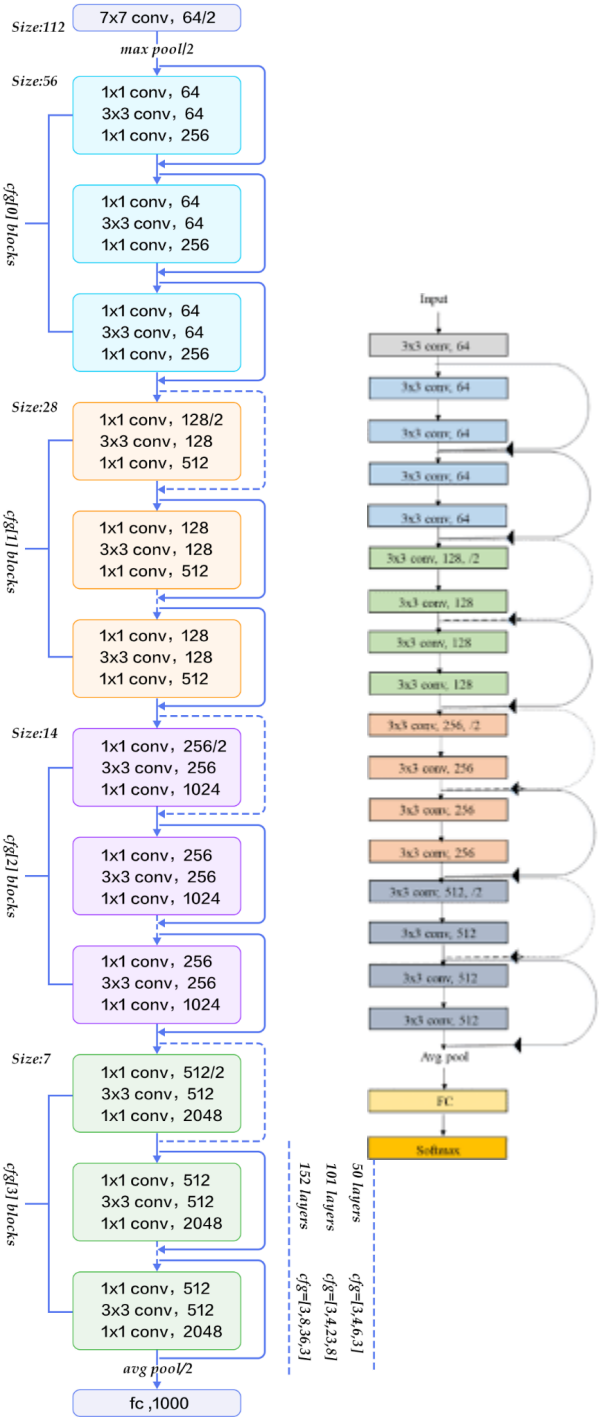
Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except † reported on the test set).

Empirical Results only further validate this.
((A) zero-padding shortcuts are used
for increasing dimensions, and all shortcuts are parameterfree (the same as Table 2 and Fig. 4 right); (B) projection shortcuts are used for increasing dimensions, and other
shortcuts are identity; and (C) all shortcuts are projections.
Table 3 in the research paper shows that all three options are considerably better than the plain counterpart. B is slightly better than A. We argue that this is because the zero-padded dimensions in A indeed have no residual learning. C is marginally better than B, and we attribute this to the extra parameters introduced by many (thirteen) projection shortcuts. But the small differences

among A/B/C indicate that projection shortcuts are not essential for addressing the degradation problem). Remarkably, although the depth is significantly increased, the 152-layer ResNet (11.3 billion FLOPs) still has lower complexity than VGG-16/19 nets (15.3/19.6 billion FLOPs).

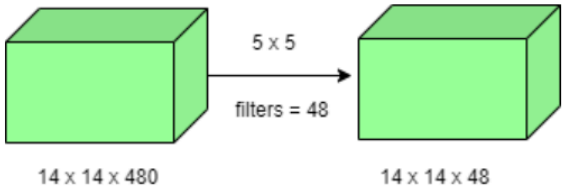
ResNet performs better because depth is actually used. Add Resnet 50, 152 structures and 18





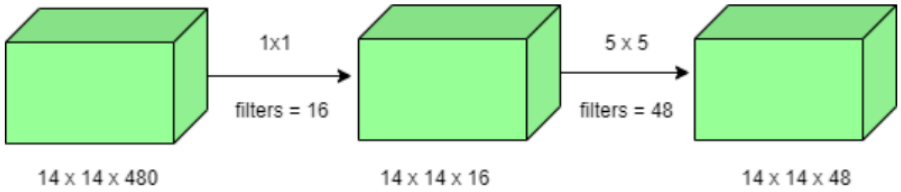
Reason for bottlenecks:

- For instance, we need to perform *5×5* convolution without using *1×1* convolution as below:



Number of operations involved here is *(14×14×48) × (5×5×480) = 112.9M*

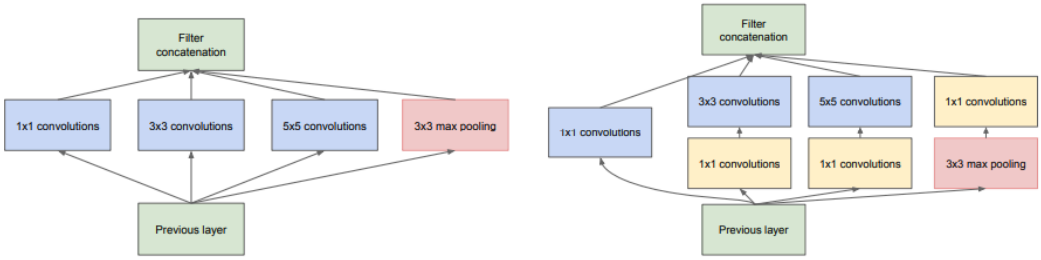
- Using 1×1 convolution:



Number of operations for 1×1 convolution = *(14×14×16) × (1×1×480) = 1.5M* Number of operations for 5×5 convolution = *(14×14×48) × (5×5×16) = 3.8M* After addition we get, *1.5M + 3.8M = 5.3M*

InceptionNet https://arxiv.org/pdf/1409.4842

The motivation behind this model was to improve the performance of deep neural networks as their size increases-depth and widthwise -- by reducing the number of parameters and computation, thus making the model smaller, less prone to overfitting and more computationally efficient.



(a) Inception module, naïve version (b) Inception module with dimension reductions

Figure 2: Inception module

Moreover, the object in the image to be classified can be present in different sizes, and selecting the right kernel size becomes a difficult choice.A large kernel size is used to capture a global distribution of the image while a small kernel size is used to capture more local information.

Inception network architecture makes it possible to use filters of multiple sizes without increasing the depth of the network. The different filters are added parallelly instead of being fully connected one after the other.The outputs of these four branches are then concatenated along the channel dimension and fed as input to the next layer. Inception introduces 1×1 convolutions as dimensionality reducers before larger convolutions, dramatically lowering the number of parameters and operations while preserving representational power.
The advancements from V1 to V3: Inception V3 is an extension of the V1 module, it uses techniques like factorizing larger convolutions to smaller convolutions (say a 5X5 convolution is factorized into two 3X3 convolutions) and asymmetric factorizations (example: factorizing a 3X3 filter into a 1X3 and 3X1 filter).
The improved use of computational resources allows for increasing both the width of each stage as well as the number of stages without getting into computational difficulties
In other words : advantage of inception net is parallel convolution enabling us to capture local and global features early on and more efficiently which in turn helps us go more deeper

These factorizations are done with the aim of reducing the number of parameters being used at every inception module. Below is an image of the inception V3 module.

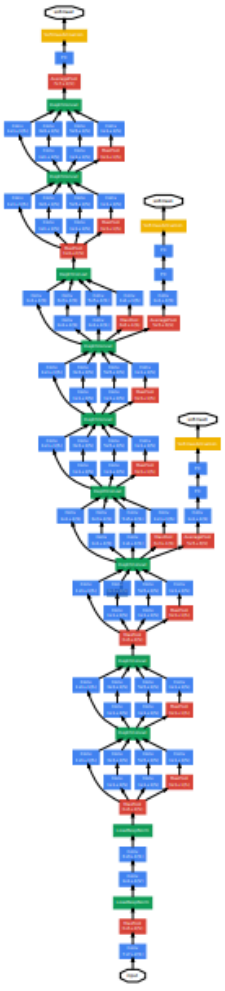


Figure 3: GoogLeNet network with all the bells and whistles

type	patch size/ stride	output size	depth	#1×1	#3×3 reduce	#3×3	#5×5 reduce	#5×5	pool proj	params	ops
convolution	7×7/2	112×112×64	1							2.7K	34M
max pool	3×3/2	56×56×64	0								
convolution	3×3/1	56×56×192	2		64	192				112K	360M
max pool	3×3/2	28×28×192	0								
inception (3a)		28×28×256	2	64	96	128	16	32	32	159K	128M
inception (3b)		28×28×480	2	128	128	192	32	96	64	380K	304M
max pool	3×3/2	14×14×480	0								
inception (4a)		14×14×512	2	192	96	208	16	48	64	364K	73M
inception (4b)		14×14×512	2	160	112	224	24	64	64	437K	88M
inception (4c)		14×14×512	2	128	128	256	24	64	64	463K	100M
inception (4d)		14×14×528	2	112	144	288	32	64	64	580K	119M
inception (4e)		14×14×832	2	256	160	320	32	128	128	840K	170M
max pool	3×3/2	7×7×832	0								
inception (5a)		7×7×832	2	256	160	320	32	128	128	1072K	54M
inception (5b)		7×7×1024	2	384	192	384	48	128	128	1388K	71M
avg pool	7×7/1	1×1×1024	0								
dropout (40%)		1×1×1024	0								
linear		1×1×1000	1							1000K	1M
softmax		1×1×1000	0								

Table 1: GoogLeNet incarnation of the Inception architecture

The number represents the “stage” or group in the architecture based on the output spatial size at that point in the network (for example, 28×28 is the third stage, so labeled as 3). The letter (a, b, etc.) distinguishes each inception module within that stage, since multiple modules are often stacked at the same resolution.

MobilNetV3 https://arxiv.org/pdf/1801.04381

A more advanced paper after ResNets, MobileNetV2’s remarkable aspect is its ability to strike a good balance between model size and accuracy, rendering it ideal for resource-constrained devices.

The architecture of MobileNet-v2 consists of a series of convolutional layers, followed by depthwise separable convolutions, inverted residuals, bottleneck design, linear bottlenecks, and squeeze-and-excitation (SE) blocks

A key part of MobileNetV2’s design is depthwise separable convolutions, a technique that was first introduced in MobileNetV1. These convolutions are used to reduce the computational load while maintaining accuracy. The convolutions are split into two steps:

Depthwise Convolution: Applies a filter to each input channel individually.
Pointwise Convolution: Uses a 1x1 convolution to combine the outputs from the depthwise step.
This separable approach reduces the number of parameters and lowers the computational cost by 8 to 9 times compared to standard convolutions, with only a small reduction in accuracy.

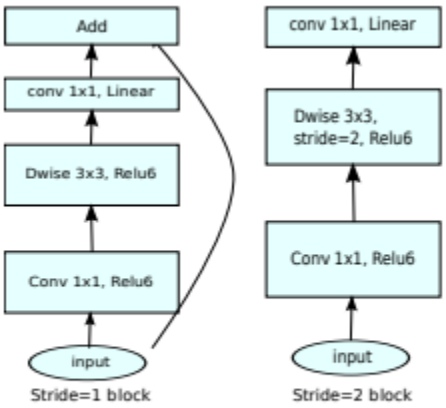
The other major contribution is the **linear bottleneck layers** - a solution to the problem of using ReLU in lower dimensions (after compressing the data) can lead to important info lost. Bottlenecks are low-dimensional layers (fewer channels) that act as compressed representations of the input.

MobileNetV2 uses linear bottlenecks (no activation after projection) to avoid losing important information during dimensionality reduction. Non-linearities like ReLU can cause information loss in low-dimensional spaces, so keeping the bottleneck linear helps maintain the integrity of the features.

Reduce Computation: By compressing the feature maps to a smaller number of channels before and after the expensive operations (like depthwise convolutions), bottlenecks reduce the number of parameters and multiply-accumulate operations, making the network more efficient for mobile and embedded devices.

Inverted Residuals Blocks: It is better to go from one bottleneck layer to another through inverted residual connections - The reason for inverted is cause of expansion of the bottle neck layers then a compression, instead of the expansion to compression seen in resnets. This benefits the model resulting in lower computation power yet giving higher accuracy because the bottlenecks actually contain all the necessary information, while an expansion layer acts merely as an implementation detail that accompanies a non-linear transformation of the tensor, we use shortcuts directly between the bottlenecks.The motivation for inserting shortcuts is similar to that of classical residual connections:we want to improve the ability of a gradient to propagate across multiplier layers

https://youtu.be/hzj9kEU8QdA?si=1aHtI-x5wroH9CgG



(d) Mobilenet V2

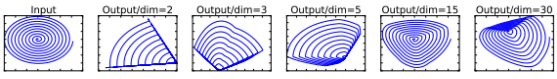


Figure 1: Examples of ReLU transformations of low-dimensional manifolds embedded in higher-dimensional spaces. In these examples the initial spiral is embedded into an n-dimensional space using random matrix T followed by ReLU, and then projected back to the 2D space using T^{-1}. In examples above $n = 2, 3$ result in information loss where certain points of the manifold collapse into each other, while for $n = 15$ to 30 the transformation is highly non-convex.

ViTs and GANs:

Helpful doc :
(CNN takes an image as input and passes it through multiple layers, gradually learning to recognize important features (e.g, in classification tasks, it learns to recognize features that distinguish one image from another by learning the kernel). Early convolutional layers detect low-level, local features like edges and corners, which are present in almost every image and are not very discriminative. Pooling layers reduce spatial dimensionality, remove redundancy, and introduce translation invariance. Deeper convolutional layers combine the earlier feature maps to form more abstract, higher-level patterns(as they are the ones that help categorise them image and capture global info) and shapes over a larger receptive field, capturing more context and more relevant features to the task.(its like u keep capturing invariant and distinguishing features (weights are learnt such that) so u finally get the most relevant set of features,Hierarchical Abstraction: As the network goes deeper, filters in higher layers combine the outputs of previous layers, enabling the network to detect more complex and abstract features (like object parts or entire objects) that are most relevant for classification.) Hierarchical because u are summarsining the entire context of the image into a smaller spatial resolution using the feature maps generated in the prev layers and since it captures the entire image it becomes more relevant? And it should become more relevant as we use these to do the classification rask.
Oh and also abstraction leads to generlaisability

The final convolutional feature maps summarize the most important features, which are then flattened and passed through fully connected layers to make the final prediction.https://arxiv.org/pdf/1311.2901 this paper states that the last layers are more visually important to humans for distinguishing.

So as cnns learn more distinguhsing features thye learn complex patterns like ears, nose, eyes etc that is actually encoded into a feature map. Instead, each feature map in deeper layers encodes a distributed pattern—a combination of many lower-level features that, together, are useful for the network's task.The most distinguishing features for a task (like car detection) may not look like a car part to us. Instead, the network learns statistical patterns that best separate cars from other objects, which can be highly abstract and not visually interpretable.)
https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=8308186&tag=1
https://poloclub.github.io/cnn-explainer/#article-convolution
https://cs231n.github.io/convolutional-networks/
https://cs231n.github.io/understanding-cnn/

The emergence of a hierarchy of features is the result of the way CNNs are laid out. Its not a coincidence.

Almost by definition, higher level concepts manifest themselves at greater spatial (e.g. in the case of images) or temporal (in the case of speech) resolution. You don't expect to identify a face in a 3x3 pixel patch, at most you may get an arbitrarily oriented edge. As you go deeper down the layers in a CNN, the receptive field of a single neuron increases. A deeper neuron is "seeing" a larger area of the input image or a larger section of a speech signal.
Also, more or less by definition, a higher level concept is one that's defined on top of lower level concepts. Letters combine to form words, which combine to form phrases, which combine to form sentences. Walls combine to form rooms, which combine to form floor plans, which combine to form buildings and so on. An elliptic curve is obtained when you combine a string of edges of smoothly varying orientations; and then a few such curves might combine to give you an "eye" or a "nose", and few such "parts" may combine to give you a face. Again, CNNs are by design laid out in a such a way, that the kth layer takes activations of the (k-1)th layer. Thus the emergence of such feature hierarchies is no accident.
Use perplexity.
https://gghantiwala.medium.com/understanding-the-architecture-of-the-inception-network-and-applying-it-to-a-real-world-dataset-169874795540
https://www.scaler.com/topics/inception-network/